CUT CLINIC • DIGITAL SERVICE IMPROVEMENT

Appointment Management System

Implemented functionality, stakeholder problems, and how the solution improves the appointment process

Stakeholder: CUT Clinic | Users: CUT students and clinic staff

2026

The stakeholder’s critical problems

The system is designed around the current clinic workflow and the delays students experience.

Requests depend on email and phone

Students contact the clinic through Outlook/email or telephone. Requests can be difficult to track consistently.

Only two staff receive requests

When those staff members are unavailable, requests can wait longer before being reviewed.

Students may miss updates

Some students have limited mobile data and cannot check Outlook regularly to see whether the clinic has responded.

Impact on the clinic and students

- **Delayed visibility:** students may not know the status of a request promptly.
- **Manual coordination:** staff must manage communication and appointment details across separate channels.
- **Need for a shared view:** staff require a clearer way to see, review, schedule, and follow up requests.

The proposed solution

One connected workflow for student requests and clinic staff decisions.

Student mobile app

Students register/sign in, submit appointment requests, view appointment status and history, and cancel when needed.

Clinic staff web portal

Staff view requests in a dashboard, review them, approve/reject, assign appointment times, and manage scheduled appointments.

Shared Supabase backend

Authentication and shared data connect both applications so changes made by staff can appear in the student app.

The key change

Move from disconnected email/phone follow-up toward a shared, status-based appointment process with a dedicated interface for students and clinic staff.

How the applications work together

High-level architecture of the implemented solution.

Flutter student app

Registration, login, requests, status and appointment history.

Supabase

Authentication + PostgreSQL data

Shared appointments, students, documents metadata and notifications.

ASP.NET Core staff portal

Staff login, dashboard, appointment handling and student details.

Data flow

1 Student submits

Request is saved to shared data.

2 Staff reviews

Request appears in the staff portal.

3 Staff decides

Approve/reject and assign a slot.

4 Student sees update

Status synchronises; email is sent for supported outcomes.

Student-facing functionality

What students can do in the Flutter application.

Account access

Register and sign in to use the appointment service.

Request an appointment

Submit a request for clinic staff to review. Staff assign the date and time.

View appointment status

See updates from the shared backend rather than relying only on email follow-up.

Appointment history

View past and current appointment information, including completed appointments.

Cancellation

Students can cancel an appointment when needed, subject to the implemented workflow.

Email updates

Receive supported appointment outcome messages by email.

Important workflow decision: students submit a request; clinic staff control approval and appointment time assignment.

Clinic staff portal functionality

The ASP.NET Core portal gives authorised clinic staff a central workspace.

Dashboard overview

Summary cards show total requests, pending, scheduled, completed, rejected, cancelled, and today's appointments.

Review requests

Open appointment requests and approve or reject them.

Scheduling controls

Assign appointment date/time and manage scheduled appointments.

Reschedule if needed

Staff can reschedule an approved appointment if the clinician becomes unavailable.

Student details

View student profile information and appointment history, including completed visits.

Email integration

Appointment emails are configured and have been confirmed to send in project testing.

Staff alerts: the in-app notification area is intended to alert staff to new appointment requests.

Appointment lifecycle

The workflow makes the clinic's decision points explicit.

1 Request submitted

Student sends request from Flutter.

2 Pending review

Request is visible in staff portal.

3A Approved + scheduled

Staff assigns date/time.

3B Rejected

Staff declines request.

4 Outcome communicated

Status appears in the apps; supported email is sent.

5 Manage appointment

Cancellation or staff rescheduling when appropriate.

Business rule: one active appointment per student; the system also checks for past appointment times and duplicate date/time bookings.

How the solution addresses stakeholder problems

The value is not just digitisation—it is a clearer process for students and staff.

Critical problem	Implemented response	Expected benefit
Requests rely on email/phone	Student app + staff portal share appointment data	One organised workflow instead of scattered follow-up
Only two staff receive emails	Requests are listed in a staff dashboard	Staff can see pending work in a central view
Students may not check Outlook daily	Status is available in the student app; supported email updates are sent	Less dependence on repeatedly checking inboxes
Scheduling is manually coordinated	Staff approve/reject and assign date/time; rescheduling is supported	Clearer decisions and fewer scheduling conflicts

These are intended operational improvements; actual reductions in delays should be measured during stakeholder testing.

Rules that improve reliability

Business rules help prevent avoidable appointment errors.

One active appointment

A student should not hold multiple active appointments at the same time.

No past appointment slots

The system checks date/time so a new booking cannot be set in the past.

Avoid duplicate slots

The appointment workflow checks for an existing booking at the same date/time.

Shared source of data

Flutter and ASP.NET read/write through the same Supabase-backed data model.

Staff authentication

Staff portal uses authenticated staff access and role-based protection.

Status visibility

Appointment statuses help staff and students understand the current state.

Technology choices and feasibility

The stack supports a student mobile interface and a staff web interface over shared data.

Flutter / Dart

Cross-platform mobile app for students.

Provides a dedicated interface for submitting and checking appointment requests.

ASP.NET Core / C#

Web portal for clinic staff.

Supports authenticated access, dashboard views, appointment management, and email integration.

Supabase / PostgreSQL

Shared backend, authentication, and structured appointment data.

Allows both apps to work from the same records.

Feasibility strengths

- **Technical:** the core mobile, web, database and email components are already working together.
- **Operational:** each user group has an interface aligned to its role.
- **Maintainability:** shared backend data reduces the need to maintain separate appointment lists.

Implemented now—and what remains

Be clear with the stakeholder about completed functionality versus future enhancements.

Implemented / demonstrated

- Flutter student app connected to Supabase
- ASP.NET Core staff login and dashboard
- Appointment request review and status updates
- Scheduling and staff rescheduling of approved appointments
- Student profile and appointment history
- Email notifications for supported appointment outcomes
- Cross-app data synchronisation
- Validation for active appointments and scheduling conflicts

Not to present as completed yet

- SMS delivery: paused; no full Twilio end-to-end test
- Registration-proof approval/rejection workflow: still to be implemented
- Production deployment and load testing: not confirmed
- Measured reduction in appointment delays: requires stakeholder evaluation

Recommended next validation: run a clinic-staff walkthrough and student scenario tests, then record feedback and evidence.

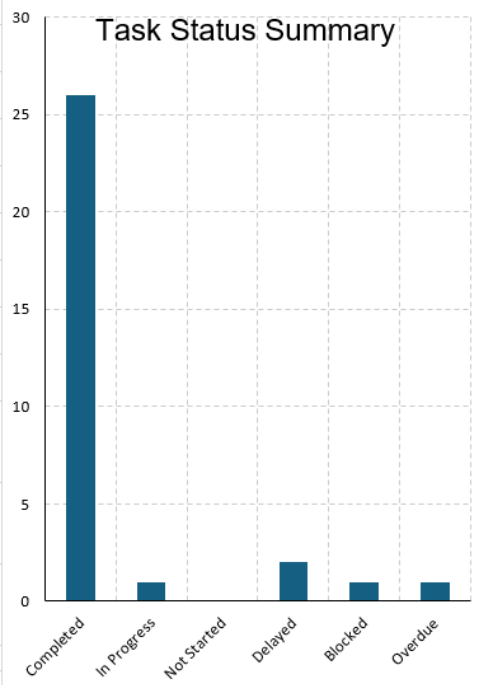
Project Tracker overview

Project Overview and Progress Dashboard

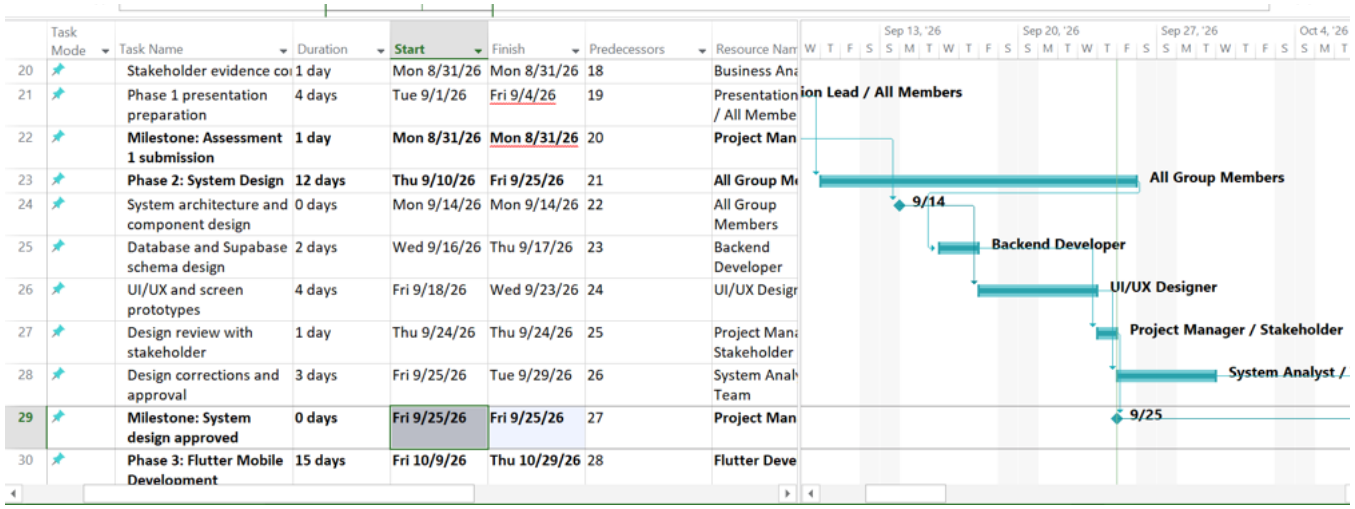
Project Information	Group Response
Group name or number	All Safe Tech
Project title	CUT Clinic Appointment Management System
Stakeholder or organisation	CUT Clinic
Group mentor	E Mokoduwe
Reporting phase	Final Phase
Reporting period	05 October 2026 - 23 October 2026
Overall status	In Progress
Overall % complete	90,0%
Main achievement	Set Up and Implement the Shared Supabase Backend, Develop the Flutter Mobile Application, Develop the ASP.NET Web Application, Integrate
Main challenge	Crushing of the PC, The messaging notification service requires a paid subscription to obtain the necessary verification code or access credentials.
Next planned activity	Complete the final phase evidence package, verify actual individual contributions and prepare for the 50% presentation/defence.

Task Status	Count
Completed	26
In Progress	1
Not Started	0
Delayed	2
Blocked	1
Overdue	1

Contribution Summary	Count
Contribution entries	25
Verified entries	25
Unverified entries	0
Stakeholder engagements	1



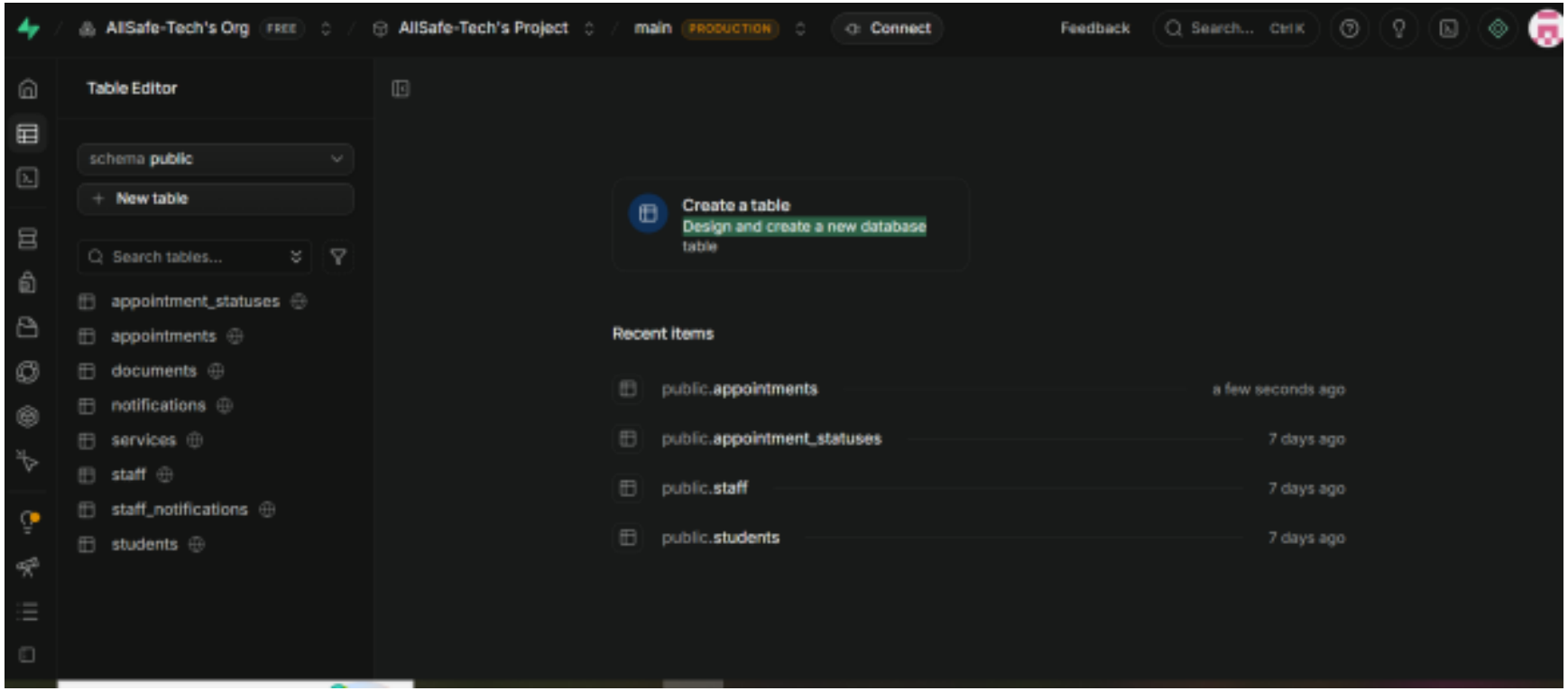
Microsoft Plan



	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Nam
20		Stakeholder evidence co	1 day	Mon 8/31/26	Mon 8/31/26	18	Business Ana
21		Phase 1 presentation preparation	4 days	Tue 9/1/26	Fri 9/4/26	19	Presentation / All Membe
22		**Milestone: Assessment 1 submission**	**1 day**	**Mon 8/31/26**	**Mon 8/31/26**	20	**Project Man**
23		**Phase 2: System Design**	**12 days**	**Thu 9/10/26**	**Fri 9/25/26**	21	**All Group M**
24		System architecture and component design	0 days	Mon 9/14/26	Mon 9/14/26	22	All Group Members
25		Database and Supabase schema design	2 days	Wed 9/16/26	Thu 9/17/26	23	Backend Developer
26		UI/UX and screen prototypes	4 days	Fri 9/18/26	Wed 9/23/26	24	UI/UX Desigr
27		Design review with stakeholder	1 day	Thu 9/24/26	Thu 9/24/26	25	Project Mana Stakeholder
28		Design corrections and approval	3 days	Fri 9/25/26	Tue 9/29/26	26	System Anal Team
29		**Milestone: System design approved**	**0 days**	**Fri 9/25/26**	**Fri 9/25/26**	27	**Project Man**
30		**Phase 3: Flutter Mobile Development**	**15 days**	**Fri 10/9/26**	**Thu 10/29/26**	28	**Flutter Deve**

Github

Supabase



https://zilvjlmoznukisaadebn.supabase.co

Suggested live demonstration

Use one realistic scenario to connect the features back to the stakeholder's problems.

1. **Student submits a request**

   Show the request being created in Flutter.

2. **Staff sees the request**

   Open the ASP.NET dashboard and point out pending requests.

3. **Staff reviews and schedules**

   Approve and assign a date/time, or demonstrate rejection.

4. **Student sees the change**

   Return to Flutter and show the updated status.

5. **Show email evidence**

   Open the test inbox and show a supported appointment email.

6. **Explain the benefit**

   Highlight less reliance on manual email checking and a central staff queue.

KEY TAKEAWAYS

A clearer appointment journey for students and clinic staff

CENTRALISED	Staff can see and manage requests in one portal.
CONNECTED	Flutter and ASP.NET use shared Supabase-backed data.
TRACEABLE	Appointment status, history and email outcomes are easier to follow.
STAKEHOLDER-FOCUSED	The design directly targets delays, fragmented communication and limited inbox access.

Questions and stakeholder feedback

CUT Clinic Appointment Management System • Flutter • ASP.NET Core • Supabase